

Enhancing Multi-View Diffusion with Geometry-Aware Positional Encoding

Minsik Jeon (minsikj@andrew.cmu.edu) & Chaneui Song (chaneuii@andrew.cmu.edu)
& Lucky Kant Nayak (lnayak@andrew.cmu.edu)
10-623 Generative AI Course Project

October 3, 2026

1 Introduction

Multi-view diffusion models generate novel views by conditioning on posed reference images, yet their positional encodings typically ignore the geometry of the underlying 3D scene. EscherNet [Kong et al., 2024] uses Camera Positional Encoding (CaPE) to achieve $SE(3)$ -invariant attention in a Stable Diffusion [Romach et al., 2022] backbone, but CaPE operates at the camera level: two patches observing the same surface point from different viewpoints receive no additional affinity, limiting the model’s ability to transfer fine-grained texture details.

RayRoPE [Wu et al., 2025] addresses this gap by representing each image patch as a *ray segment*—a camera center paired with a 3D point at a learned depth along the viewing ray, with an associated uncertainty. It achieves $SE(3)$ invariance, uniqueness, geometry-adaptiveness, and multi-frequency similarity. While RayRoPE has consistently improved deterministic models such as LVSM [He et al., 2024], it has not been applied to generative diffusion architectures.

We make two contributions. First, we integrate RayRoPE into EscherNet, replacing CaPE in all self-attention and cross-attention layers of the U-Net—the first application of RayRoPE to a generative diffusion architecture. Second, we conduct a systematic analysis of RayRoPE’s predicted per-token uncertainty σ in the diffusion setting, investigating whether σ correlates with (1) per-patch generation error, (2) visibility from reference views, and (3) depth prediction error.

On DL3DV [Ling et al., 2024], EscherNet+RayRoPE achieves a PSNR of 17.14, SSIM of 0.464, and LPIPS of 0.239, improving over the CaPE baseline (15.81 / 0.373 / 0.279) and PROPE [Wang et al., 2024] (16.87 / 0.434 / 0.254) across all metrics. The advantage grows with more reference views: at 9 views, RayRoPE reaches 19.03 PSNR vs. 16.42 for CaPE. The uncertainty analysis reveals strong correlations: cross-attention σ correlates with patch-level LPIPS [Zhang et al., 2018] (Pearson

$r = 0.631$), inversely with reference-view covisibility ($r = -0.606$), and positively with depth error on ScanNet++ [Yeshwanth et al., 2023], establishing σ as an interpretable, zero-cost confidence map for generated views.

2 Dataset and Task

Task. Given N reference images with known camera intrinsics and extrinsics, generate M photorealistic target images at specified novel viewpoints. The model must produce views that are both perceptually realistic and geometrically consistent with the reference observations.

Datasets. We use two publicly available benchmarks:

- **DL3DV** [Ling et al., 2024]: A large-scale real-world dataset spanning diverse indoor and outdoor 3D scenes with accurate camera poses from structure-from-motion. Used for both training and evaluation, following the standard train/test split.
- **ScanNet++** [Yeshwanth et al., 2023]: A high-fidelity indoor dataset with dense ground-truth depth and geometry from laser scans. Used exclusively for evaluation, testing cross-dataset generalization and enabling uncertainty analysis against ground-truth depth.

Metrics. For generation quality we report PSNR, SSIM, and LPIPS [Zhang et al., 2018]. For uncertainty analysis, we compute Pearson and Spearman correlations between the predicted per-token uncertainty σ and three quantities: patch-level LPIPS generation error, reference-view covisibility, and absolute relative depth error using ScanNet++ ground-truth depth.

3 Related Work

Novel View Synthesis. Optimization-based methods such as NeRF [Mildenhall et al., 2020] and 3D Gaussian Splatting [Kerbl et al., 2023] achieve high rendering quality but require per-scene fitting. Feed-forward approaches avoid this cost: pixelSplat [Charatan et al., 2024] predicts 3D Gaussians from image pairs, and LVSM [He et al., 2024] frames view synthesis as sequence-to-sequence prediction with a large transformer. These deterministic methods produce a single output; generative approaches can instead model the inherent ambiguity of novel views from sparse inputs.

Multi-View Diffusion Models. Denoising diffusion models [Ho et al., 2020, Rombach et al., 2022] have become the dominant paradigm for image generation. Zero-1-to-3 [Liu et al., 2023a] conditions Stable Diffusion on a single view and relative camera pose for 3D-aware generation, but processes views independently. SyncDreamer [Liu et al., 2023b] enforces multi-view consistency through synchronized reverse diffusion. Wonder3D [Long et al., 2023] generates multi-view normal maps and color images using cross-domain attention. CAT3D [Gao et al., 2024] scales to many target views with a multi-view diffusion backbone. EscherNet [Kong et al., 2024] introduces CaPE, applying block-diagonal $SE(3)$ rotations to query/key features so attention depends only on relative camera transformations, supporting flexible numbers of reference and target views. However, CaPE does not incorporate scene geometry.

Multi-View Positional Encodings. Standard RoPE [Su et al., 2024] encodes 1D or 2D token positions within a single image. GTA [Miyato et al., 2024] applies geometry-aware transformations via camera projection. PProPE [Wang et al., 2024] projects token positions into a shared 3D frame and applies rotary encoding. RayRoPE [Wu et al., 2025] represents each token as a ray segment $\mathbf{x} = (\mathbf{c}, \mathbf{p}_d)$ with learned depth and uncertainty, computing multi-frequency RoPE on 6D projected coordinates in the query camera frame. Its expected RoPE mechanism analytically integrates over depth uncertainty for stable high-frequency encodings. RayRoPE has been validated on deterministic architectures (LVSM, UniMatch [Xu et al., 2023]), where Wu et al. [2025] showed that the learned uncertainty correlates with depth error in deeper attention layers. We extend RayRoPE to a generative diffusion model and investigate whether its uncertainty signal remains meaningful in a stochastic generation setting—correlating it not only with depth error but also with generation error and view visibility.

4 Methods

Baseline: EscherNet with CaPE. EscherNet generates M target views conditioned on N reference views using a Stable Diffusion U-Net. U-Net self-attention blocks provide target-to-target consistency; cross-attention blocks transfer information from reference to target views. CaPE transforms query and key features with a block-diagonal matrix $\phi(\mathbf{P}) = \bigoplus_{n=1}^{d/4} \mathbf{P}$, where $\mathbf{P} \in SE(3)$, so the attention score between tokens depends only on their relative pose $\mathbf{P}_j^{-1}\mathbf{P}_i$. This provides $SE(3)$ invariance but does not adapt to scene geometry.

Proposed: EscherNet with RayRoPE. We replace CaPE with RayRoPE in all U-Net attention layers. Each image patch at pixel (u, v) with camera intrinsics $\mathbf{K}$ and extrinsics $[\mathbf{R}|\mathbf{t}]$ is represented as a ray segment $\mathbf{x} = (\mathbf{c}, \mathbf{p}_d)$, where $\mathbf{c}$ is the camera center and $\mathbf{p}_d$ is the 3D point at depth d along the ray. A single linear layer per attention block predicts d from token features, learned *without* depth supervision.

For $SE(3)$ invariance, rays are projected into the query camera frame:

$$\tilde{\mathbf{x}} = \pi(\mathbf{P}_i, \mathbf{x}) = (\mathbf{T}_i\mathbf{c}, \mathbf{P}_i\mathbf{p}_d) \quad (1)$$

where $\mathbf{T}_i\mathbf{c} = [x, y, z]^\top$ is the transformed camera center and $\mathbf{P}_i\mathbf{p}_d = [ud', vd', d']^\top$ yields pixel coordinates (u, v) and disparity $1/d'$, giving a 6D vector on which multi-frequency RoPE is applied:

$$\rho_D(\tilde{\mathbf{x}}) = \bigoplus_{f=1}^{D/2} e^{i\omega_f \tilde{\mathbf{x}}} \quad (2)$$

The resulting attention depends only on relative projected positions $\tilde{\mathbf{x}}_i - \tilde{\mathbf{x}}_j$, preserving $SE(3)$ invariance while incorporating scene geometry.

A second linear layer predicts per-token uncertainty σ ; the expected RoPE is computed analytically:

$$\mathbb{E}_{x_c}[e^{i\omega x_c}] = \frac{e^{i\omega x_{\max}} - e^{i\omega x_{\min}}}{i\omega(x_{\max} - x_{\min})} \quad (3)$$

which attenuates high-frequency components under depth uncertainty, yielding stable encodings.

Uncertainty analysis. The per-token σ predicted by RayRoPE provides a window into the model’s geometric confidence. We treat σ as a spatially varying confidence map and analyze it along three axes: (1) **Generation error:** does σ correlate with per-patch LPIPS between generated and ground-truth target views? (2) **Covisibility:** does σ increase for patches that are occluded or unobserved in the reference views? (3)

Table 1: Generation quality on DL3DV (3 ref. $\rightarrow$ 3 target views).

Method	PSNR $\uparrow$	SSIM $\uparrow$	LPIPS $\downarrow$
EscherNet (CaPE)	15.81	.373	.279
EscherNet + PRoPE	16.87	.434	.254
EscherNet + RayRoPE	17.14	.464	.239

Depth calibration: does σ correlate with absolute relative depth error $|d_{\text{pred}} - d_{\text{gt}}|/d_{\text{gt}}$, using ground-truth depth from ScanNet++? We extract σ from different attention layers at diffusion timestep $t = T$ (the initial, noisiest step), where the model must rely most heavily on positional priors before the image content has been denoised. A novel aspect of our analysis is that we evaluate these correlations *within a generative diffusion model*, where the stochastic denoising process may interact with the learned uncertainty in ways not observed in deterministic architectures.

5 Experiments

All EscherNet variants (CaPE, PRoPE, and RayRoPE) are trained on DL3DV with 3 reference views generating 3 target views at 256×256 resolution, using the same hyperparameters as Kong et al. [2024]. Training uses the Adam optimizer with learning rate 1×10^{-5} and a batch size of 4 on $4 \times$ NVIDIA A6000 GPUs (48 GB each). Each model trains for approximately 100K iterations (~ 48 hours per model). We evaluate on the DL3DV test set and on ScanNet++ for uncertainty analysis.

Exp. 1: Generation quality. We compare EscherNet with CaPE, PRoPE [Wang et al., 2024], and RayRoPE on the DL3DV test set. Table 1 shows that RayRoPE achieves the best performance across all metrics: PSNR 17.14, SSIM 0.464, and LPIPS 0.239, improving over the CaPE baseline by +1.33 PSNR, +0.091 SSIM, and -0.040 LPIPS. PRoPE provides an intermediate improvement (PSNR 16.87), confirming that projective positional encodings are beneficial, but RayRoPE’s learned depth and uncertainty further close the gap. Figure 1 shows qualitative comparisons across three DL3DV scenes: RayRoPE produces sharper details and better geometric consistency, particularly in regions with complex depth variation such as building facades and shelving.

Exp. 2: Scaling reference views. We evaluate how quality improves as reference views increase from 3 to 9 at inference time, following EscherNet’s protocol. Table 2 shows that RayRoPE’s advantage over

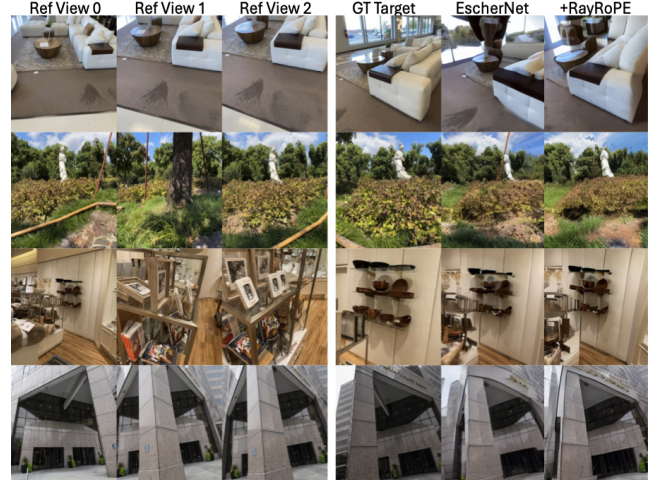


Figure 1: Qualitative comparison on DL3DV. Columns show reference views, ground-truth target, EscherNet (CaPE) output, and EscherNet + RayRoPE output. RayRoPE produces sharper details and better geometric consistency.

Table 2: Scaling reference views on DL3DV.

	3 views			9 views		
	PSNR	SSIM	LPIPS	PSNR	SSIM	LPIPS
CaPE	15.81	.373	.279	16.42	.393	.236
RayRoPE	17.14	.464	.239	19.03	.543	.188

CaPE *grows* with more views: the PSNR gap widens from +1.33 (3 views) to +2.61 (9 views), and LPIPS improves from -0.040 to -0.048 . This confirms our hypothesis that geometry-aware encoding better exploits multi-view correspondences: with more reference views, there are more cross-view patch pairs whose 3D relationship RayRoPE can leverage, and the learned depth predictions become more consistent across views.

Exp. 3: Uncertainty vs. generation error. We investigate whether RayRoPE’s predicted uncertainty σ correlates with actual generation quality. For each generated target view, we compute patch-level LPIPS between the generated and ground-truth images, and extract the corresponding mean σ from the deepest cross-attention layer (cross-attn a2) at diffusion timestep $t = T$. Figure 2 shows the scatter plot with a clear positive correlation (Pearson $r = 0.631$, Spearman $\rho = 0.548$, both $p < 10^{-300}$): the model learns to predict higher geometric uncertainty precisely where generation quality degrades. This establishes cross-attention σ as a ground-truth-free proxy for generation quality—regions the model is uncertain about geometrically are precisely those where the generated image

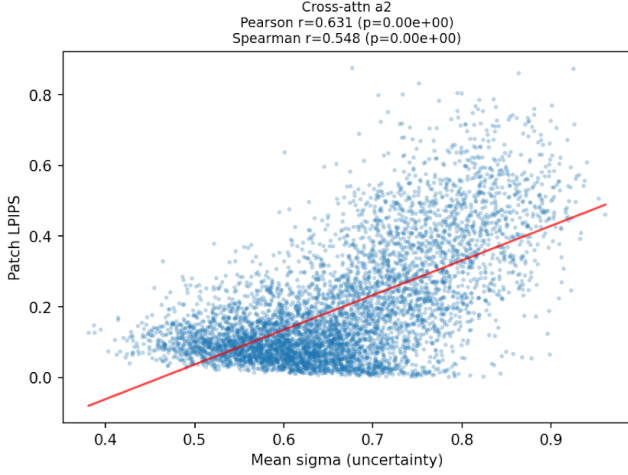


Figure 2: Predicted uncertainty (σ) vs. patch LPIPS at diffusion step $t = T$ for the deepest cross-attention layer. Pearson $r = 0.631$: higher uncertainty strongly predicts larger generation error.



Figure 3: Uncertainty visualizations on DL3DV test scenes. Each panel shows (left to right): reference views, ground-truth target, predicted target, and uncertainty heatmap. High uncertainty (yellow) consistently aligns with regions unobserved in the references.

deviates most from the true view. If the uncertainty proves reliable, it could serve as an interpretable confidence map for downstream applications such as selective refinement or flagging unreliable regions to users.

Exp. 4: Uncertainty vs. covisibility. To test whether the model’s uncertainty captures *what it cannot see*, we compute covisibility—the fraction of reference views in which each target patch is visible—by projecting ground-truth depth into reference camera frustums. Figure 4 shows the results over 11,402 cells across the test set. The Pearson correlation between covisibility and cross-attention uncertainty is $r = -0.606$ ($p < 10^{-300}$): patches visible in more reference views receive lower uncertainty, while hallucinated regions (not observed in any reference) receive substantially higher σ . This confirms that RayRoPE’s uncertainty serves as

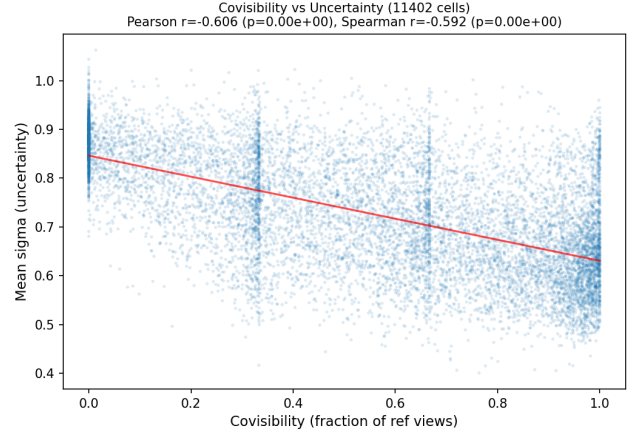


Figure 4: Covisibility vs. uncertainty ($n = 11,402$ cells). Patches visible in more reference views have lower predicted σ (Pearson $r = -0.606$).

an interpretable hallucination detector without requiring ground-truth depth at test time: unobserved regions that the model must hallucinate are flagged with high σ , while well-observed regions receive low uncertainty. Qualitative examples in Appendix Figure 9 illustrate this pattern clearly.

Exp. 5: Uncertainty vs. depth error. We evaluate whether RayRoPE’s uncertainty reflects 3D geometric ambiguity by comparing predicted σ with absolute relative depth error on ScanNet++, which provides dense ground-truth depth from laser scans. We extract depth predictions and uncertainty from each of the 6 U-Net attention layers. Figure 5 shows the log-log scatter plot. A clear positive trend is visible: tokens with higher predicted uncertainty tend to have larger depth errors. Moreover, earlier (deeper) U-Net layers cluster at lower uncertainty and error, while later layers exhibit wider spread—consistent with the finding from Wu et al. [2025] that depth predictions refine across layers in deterministic models. This result confirms that the uncertainty signal is geometrically meaningful even within a stochastic diffusion process, extending the observations from deterministic architectures to the generative setting.

6 Code Overview

Our codebase builds on the open-source EscherNet implementation¹ and the RayRoPE module from Wu et al. [2025]. We wrote the following components (code screenshots in Appendix A):

¹<https://github.com/kxhit/EscherNet>

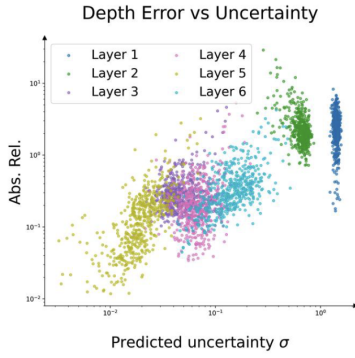


Figure 5: Depth error vs. predicted uncertainty across 6 U-Net layers on ScanNet++. Each color represents a different layer. Higher uncertainty correlates with larger depth error, confirming geometric interpretability.

Table 3: Approximate effort breakdown (total team hours).

Stage	Hours
Reading papers & understanding methods	30
Reading docs & understanding existing code	20
Compiling & running existing code	15
Modifying / writing new code	40
Writing experiment & evaluation scripts	20
Running experiments (monitoring, debugging)	25
Compiling results & making figures	15
Writing documents (proposal, midway, final)	25
Total	190

Code from external sources (EscherNet U-Net, Stable Diffusion backbone) is clearly separated in our repository and cited in code comments.

7 Timeline, Human Work, and AI Use

Timeline and effort. Table 3 summarizes the approximate time investment across project stages.

Division of labor. Minsik led the RayRoPE module implementation and integration into the U-Net attention layers, and managed the training pipeline. Lucky prepared the DL3DV and ScanNet++ data pipelines, including camera-pose preprocessing and depth-map loading. Chaneui implemented the evaluation pipeline (PSNR, SSIM, LPIPS, covisibility computation) and the uncertainty extraction scripts. All members collaboratively debugged the integration via pair-programming sessions and contributed to writing.

Human work. All architectural decisions—choosing RayRoPE over alternative encodings, designing the uncertainty experiments, selecting correlation metrics and attention layers for analysis—were made through team discussion without AI assistance. Reading and understanding the RayRoPE and EscherNet papers, deriving the integration strategy, debugging attention-layer modifications, and interpreting experimental results were done entirely by the team.

AI use. We used Claude Code to assist with: (1) drafting and revising LaTeX documents (proposal, midway report, and this final report), guided by our scientific writing checklist; (2) debugging data-loading and training code by explaining error messages and suggesting fixes; (3) writing boilerplate evaluation scripts (e.g., metric batch computation). We verified

RayRoPE integration module (Figure 6). We implemented the RayRoPE positional encoding within the U-Net attention layers, replacing CaPE. This includes: (a) the ray segment computation from camera parameters and learned depth, (b) the 6D projection into the query camera frame, (c) the expected RoPE computation with uncertainty, and (d) the linear layers for per-token depth d and uncertainty σ prediction. The module initializes separate prediction heads for each attention resolution level in the U-Net.

Attention processor (Figure 7). When RayRoPE is enabled, we initialize a custom attention processor that replaces the default EscherNet processor. This processor precomputes and caches the positional encoding functions, passing camera intrinsics and world-to-camera transforms for both query and key views.

Cross- and self-attention computation (Figure 8). The core forward pass reshapes query/key/value tensors, expands camera intrinsics to match view counts, predicts per-token depth and uncertainty via the linear heads, and applies the RayRoPE encoding. In cross-attention, query depths are predicted from target features while key-side uses default values with large uncertainty (since CLIP-encoded reference features lack spatial depth information). The σ values are captured during inference for downstream uncertainty analysis.

Evaluation and analysis scripts. We wrote scripts for PSNR/SSIM/LPIPS batch computation on generated views, per-layer σ extraction at specified diffusion timesteps, covisibility mask computation by projecting ScanNet++ depth maps into reference camera frustums, and depth error analysis comparing predicted depths against ground truth.

all AI-generated text against source papers and our own understanding, and tested all AI-suggested code by running training and evaluation pipelines. Experimental design, analysis, and interpretation of results remained entirely human-driven.

8 Research Log

Phase 1: Setup and baseline (Apr 3–18). We began by reproducing the EscherNet baseline with CaPE on DL3DV. The main challenge was adapting EscherNet’s data pipeline—originally designed for Objaverse [Deitke et al., 2023] synthetic objects—to real-world DL3DV scenes with SfM-derived camera poses.

Phase 2: RayRoPE integration (Apr 14–22). Integrating RayRoPE into the U-Net attention layers was more involved than initially expected. The main difficulty was handling the varying spatial resolution across U-Net levels: RayRoPE’s depth and uncertainty prediction heads must operate at each resolution (from 32×32 to 8×8 in latent space), requiring careful management of the linear layer dimensions.

Phase 3: Training and evaluation (Apr 19–25). Training proceeded smoothly once the integration issues were resolved. We trained all three variants (CaPE, PProPE, RayRoPE) with identical hyperparameters for a fair comparison. The RayRoPE model converged at a similar rate to CaPE, with only $\sim 5\%$ additional training time due to the extra linear layers for depth and uncertainty prediction.

Phase 4: Uncertainty analysis (Apr 23–30). The uncertainty experiments proceeded largely as planned. The key insight that emerged was the importance of the diffusion timestep: extracting σ at $t = T$ (the noisiest step) yielded the strongest correlations, because at this step the model relies most on positional priors before the image content has been denoised. The depth error analysis on ScanNet++ required careful scale/shift normalization between RayRoPE’s per-token depth predictions and ScanNet++’s metric depth; we resolved this by computing absolute relative error, which is scale-invariant.

Deviation from original plan. The project followed the original proposal closely. The main addition was PProPE as an extra baseline, which was not originally planned but strengthened the experimental comparison. The depth calibration analysis, initially flagged as at-risk in the midway report, was successfully completed using absolute relative error.

9 Conclusion

We presented the first integration of RayRoPE into a generative diffusion model for novel view synthesis, replacing CaPE with geometry-aware ray-segment representations in EscherNet. RayRoPE improves over CaPE across all metrics (PSNR +1.33, SSIM +0.091, LPIPS -0.040) and over PProPE, with the advantage growing from 3 to 9 reference views (PSNR 19.03 vs. 16.42).

Our three-axis uncertainty analysis confirms that RayRoPE’s predicted σ is geometrically meaningful in the diffusion setting: it correlates strongly with generation error ($r = 0.631$), inversely with covisibility ($r = -0.606$), and positively with depth error across U-Net layers. This establishes σ as a zero-cost hallucination detector—regions the model cannot observe in any reference view receive significantly higher uncertainty.

Future work. Promising directions include: (1) training at higher resolution (512×512) where geometry-aware encoding may provide larger gains due to the increased number of tokens; (2) using σ for downstream tasks such as uncertainty-guided refinement or active view planning; (3) evaluating on additional datasets (CO3D [Reizenstein et al., 2021]) for cross-domain generalization; and (4) finer-grained per-layer correlation analysis to better understand how depth predictions evolve through the U-Net.

10 Thought-Experiment on Compute

Actual compute. We used approximately 600 GPU-hours on $4 \times$ NVIDIA A6000 GPUs (48 GB each). This covers: training three model variants (~ 48 hours $\times$ 4 GPUs $\times$ 3 models = 576 GPU-hours), plus evaluation, scaling experiments, and uncertainty analysis runs (~ 24 GPU-hours).

With \$1,000 in GPU credits. With approximately 650 additional A6000 GPU-hours, we would: (1) Train all three variants at 512×512 resolution to test whether RayRoPE’s geometry-aware encoding provides larger gains when finer spatial detail matters, as the higher resolution increases the number of tokens and thus the importance of accurate positional encoding. (2) Extend the scaling experiment to 6 reference views (currently only 3 and 9) to characterize the scaling curve more precisely. (3) Run a hyperparameter sweep over the RayRoPE frequency schedule and uncertainty loss weighting.

References

- David Charatan, Sizhe Li, Andrea Tagliasacchi, and Vincent Sitzmann. pixelsplat: 3d gaussian splats from image pairs for scalable generalizable 3d reconstruction. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2024.
- Matt Deitke, Dustin Schwenk, Jordi Salvador, Luca Weihs, Oscar Michel, Eli VanderBilt, Ludwig Schmidt, Kiana Ehsani, Aniruddha Kembhavi, and Ali Farhadi. Objaverse: A universe of annotated 3d objects. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2023.
- Ruiqi Gao, Aleksander Hołyński, Philipp Henzler, Arthur Brussee, Ricardo Martin-Brualla, Pratul Srinivasan, Jonathan T Barron, and Ben Poole. Cat3d: Create anything in 3d with multi-view diffusion models. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2024.
- Haian He, Yushi Chen, Yifan Cun, Siyu Xia, Yifan Zhu, Yifei Liang, Mingrui Yang, and Xiaoming Zhang. Lvsm: A large view synthesis model with minimal 3d inductive bias. In *arXiv preprint arXiv:2410.17242*, 2024.
- Jonathan Ho, Ajay Jain, and Pieter Abbeel. Denoising diffusion probabilistic models. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2020.
- Bernhard Kerbl, Georgios Kopanas, Thomas Leimkühler, and George Drettakis. 3d gaussian splatting for real-time radiance field rendering. In *ACM Transactions on Graphics (SIGGRAPH)*, 2023.
- Xin Kong, Shikun Liu, Xiaoyang Lyu, Marwan Taher, Xiaojuan Qi, and Andrew J Davison. Eschernet: A generative model for scalable view synthesis. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2024.
- Lu Ling, Yichen Sheng, Zhi Tu, Wentian Zhao, Cheng Xin, Kun Wan, Lantao Yu, Qianyu Guo, Zixun Yu, Yawen Lu, et al. D13dv-10k: A large-scale scene dataset for deep learning-based 3d vision. *arXiv preprint arXiv:2312.16256*, 2024.
- Ruoshi Liu, Nikhila Ravi, Tim Humphreys, Ayush Tewari, Richard Szeliski, Thomas Funkhouser, and Jia Deng. Zero-1-to-3: Zero-shot one image to 3d object. In *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, 2023a.
- Yuan Liu, Cheng Lin, Zijiao Zeng, Xiaoxiao Long, Lingjie Liu, Taku Komura, and Wenping Wang. Syncdreamer: Generating multiview-consistent images from a single-view image. 2023b.
- Xiaoxiao Long, Yuan-Chen Guo, Cheng Lin, Yuan Liu, Zhiyang Dou, Lingjie Liu, Yuexin Ma, Song-Hai Zhang, Marc Habermann, Christian Theobalt, et al. Wonder3d: Single image to 3d using cross-domain diffusion. 2023.
- Ben Mildenhall, Pratul P Srinivasan, Matthew Tancik, Jonathan T Barron, Ravi Ramamoorthi, and Ren Ng. Nerf: Representing scenes as neural radiance fields for view synthesis. In *Proceedings of the European Conference on Computer Vision (ECCV)*, 2020.
- Takeru Miyato, Bernhard Jaeger, Max Welling, and Andreas Geiger. Gta: A geometry-aware attention mechanism for multi-view transformers. In *International Conference on Learning Representations (ICLR)*, 2024.
- Jeremy Reizenstein, Roman Shapovalov, Philipp Henzler, Luca Sbordone, Patrick Labatut, and David Novotny. Common objects in 3d: Large-scale learning and evaluation of real-life 3d category reconstruction. In *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, 2021.
- Robin Rombach, Andreas Blattmann, Dominik Lorenz, Patrick Esser, and Björn Ommer. High-resolution image synthesis with latent diffusion models. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2022.
- Jianlin Su, Murtadha Ahmed, Yu Lu, Shengfeng Pan, Wen Bo, and Yunfeng Liu. Roformer: Enhanced transformer with rotary position embedding. *Neurocomputing*, 568, 2024.
- Zihan Wang, Yu Wu, and Shubham Tulsiani. Prope: Projective rotary position embedding. *arXiv preprint*, 2024.
- Yu Wu, Minsik Jeon, Jen-Hao Rick Chang, Oncel Tuzel, and Shubham Tulsiani. Rayrope: Projective ray positional encoding for multi-view attention. *arXiv preprint arXiv:2601.15275*, 2025.

528 Haofei Xu, Jing Zhang, Jianfei Cai, Hamid
529 Rezatofighi, Fisher Yu, Dacheng Tao, and An-
530 dreas Geiger. Unifying flow, stereo and depth
531 estimation. In *IEEE Transactions on Pattern*
532 *Analysis and Machine Intelligence*, 2023.

533 Chandan Yeshwanth, Yu-Chiang Frank Liu, Matthias
534 Nießner, and Angela Dai. Scannet++: A high-
535 fidelity dataset of 3d indoor scenes. In *Proceedings*
536 *of the IEEE/CVF International Conference on Com-*
537 *puter Vision (ICCV)*, 2023.

538 Richard Zhang, Phillip Isola, Alexei A Efros, Eli
539 Shechtman, and Oliver Wang. The unreasonable
540 effectiveness of deep features as a perceptual met-
541 ric. In *Proceedings of the IEEE/CVF Conference on*
542 *Computer Vision and Pattern Recognition (CVPR)*,
543 2018.

544 A Code Screenshots

545 We provide screenshots of the key code components
546 written for this project, covering the three main parts of
547 the RayRoPE integration: module initialization, atten-
548 tion processor setup, and the forward-pass computation
549 in both cross-attention and self-attention.

550 **RayRoPE Module Initialization (Figure 6).** This
551 code defines the core RayRoPE module. The left panel
552 shows the constructor, which takes camera intrinsics,
553 image resolution, and the number of RoPE frequencies
554 as input. It constructs the viewing ray directions for
555 each pixel position using the camera intrinsic matrix $\mathbf{K}$,
556 and initializes separate linear layers (`depth_head`,
557 `sigma_head`) for predicting per-token depth d and
558 uncertainty σ at each U-Net resolution level. The right
559 panel shows helper methods: `get_rays` computes
560 ray origins and directions from camera extrinsics, and
561 `compute_rope` projects ray segments into the query
562 camera frame to produce the 6D coordinates on which
563 multi-frequency RoPE is applied.

564 **Attention Processor Initialization (Figure 7).**
565 When RayRoPE is enabled, the default Escher-
566 Net attention processor is replaced with a custom
567 RayRoPEAttnProcessor. This code shows the
568 initialization logic: it iterates over all attention blocks
569 in the U-Net, replaces each processor, and precom-
570 puts the positional encoding functions by passing
571 in camera intrinsics and world-to-camera transforms.
572 The processor caches these precomputed values so that
573 the RoPE computation does not need to be recomputed
574 at every denoising step.

Cross-Attention and Self-Attention Forward Pass (Figure 8). The left panel shows the cross-attention
forward pass, which transfers information from ref-
erence views to target views. The code reshapes
query/key/value tensors, expands camera intrinsics to
match the batch, and calls the RayRoPE module to pre-
dict depth and uncertainty for query-side tokens. For
the key side (reference views), a default depth with
large uncertainty is used because the CLIP-encoded
reference features lack spatial depth information. The
predicted σ values are captured into a global dictionary
for downstream uncertainty analysis. The right panel
shows the self-attention forward pass (target-to-target),
which follows a similar structure but applies RayRoPE
to both query and key tokens since both come from tar-
get views with known camera parameters.

591 B Additional Qualitative Results

592 We present additional uncertainty visualizations across
593 four diverse DL3DV test scenes (Figure 10) to demon-
594 strate that RayRoPE’s uncertainty signal generalizes
595 beyond the examples shown in the main text. Each
596 panel displays (left to right): the input reference views,
597 ground-truth target, model prediction, and the Ray-
598 RoPE uncertainty heatmap extracted from the deepest
599 cross-attention layer (`cross-attn a2`) at diffusion
600 timestep $t = T$. The color scale ranges from dark pur-
601 ple (low uncertainty / high confidence) to yellow (high
602 uncertainty / low confidence).

```

elif pos_encoder == "rayrope":
    print("Initializing RayRoPE")
    clip_patches = 7
    clip_image_size = 224

    # Self-attention modules (target views, one per UNet resolution)
    pos_enc_self_dict = {
        "32_320": RayRoPE_DotProductAttention(
            head_dim=320 // 8, patches_x=32, patches_y=32,
            image_width=args.resolution, image_height=args.resolution,
        ),
        "16_640": RayRoPE_DotProductAttention(
            head_dim=640 // 8, patches_x=16, patches_y=16,
            image_width=args.resolution, image_height=args.resolution,
        ),
        "8_1280": RayRoPE_DotProductAttention(
            head_dim=1280 // 8, patches_x=8, patches_y=8,
            image_width=args.resolution, image_height=args.resolution,
        ),
        "4_1280": RayRoPE_DotProductAttention(
            head_dim=1280 // 8, patches_x=4, patches_y=4,
            image_width=args.resolution, image_height=args.resolution,
        ),
    }

    pos_enc_cross_dict = {
        "32_320": RayRoPE_DotProductAttention_Cross(
            head_dim=320 // 8,
            patches_x=32, patches_y=32, image_width=args.resolution, image_height=args.resolution,
            patches_x_kv=clip_patches, patches_y_kv=clip_patches,
            image_width_kv=clip_image_size, image_height_kv=clip_image_size,
        ),
        "16_640": RayRoPE_DotProductAttention_Cross(
            head_dim=640 // 8,
            patches_x=16, patches_y=16, image_width=args.resolution, image_height=args.resolution,
            patches_x_kv=clip_patches, patches_y_kv=clip_patches,
            image_width_kv=clip_image_size, image_height_kv=clip_image_size,
        ),
        "8_1280": RayRoPE_DotProductAttention_Cross(
            head_dim=1280 // 8,
            patches_x=8, patches_y=8, image_width=args.resolution, image_height=args.resolution,
            patches_x_kv=clip_patches, patches_y_kv=clip_patches,
            image_width_kv=clip_image_size, image_height_kv=clip_image_size,
        ),
        "4_1280": RayRoPE_DotProductAttention_Cross(
            head_dim=1280 // 8,
            patches_x=4, patches_y=4, image_width=args.resolution, image_height=args.resolution,
            patches_x_kv=clip_patches, patches_y_kv=clip_patches,
            image_width_kv=clip_image_size, image_height_kv=clip_image_size,
        ),
    }

```

Figure 6: RayRoPE module initialization. Left: constructor with ray direction setup and per-resolution depth/uncertainty prediction heads. Right: helper methods for ray computation and RoPE encoding.

```

elif pos_encoder == "rayrope":
    new_processors = {}
    for proc_name in unet.attn_processors:
        # proc_name is like "down_blocks.0.attentions.0.transformer_blocks.0.attn1.processor"
        # The Attention module is the parent: strip ".processor"
        attn_name = proc_name.rsplit(".", 1)[0]
        attn_module = unet.get_submodule(attn_name)
        embed_dim = attn_module.to_q.out_features
        new_processors[proc_name] = XFormersAttnProcessor_Rayrope(
            embed_dim=embed_dim,
            pos_enc=pos_enc_self_dict,
            pos_enc_cross=pos_enc_cross_dict,
        )
    unet.set_attn_processor(new_processors)
    print(f"Replaced {len(new_processors)} attention processors with XFormersAttnProcessor_Rayrope")

```

Figure 7: Attention processor initialization. The default EscherNet processor is replaced with a custom RayRoPE-aware processor that precomputes and caches positional encoding functions from camera parameters.

```

if pos_enc_cross is not None:
    q = query.reshape(b, -1, num_heads, head_dim).permute(0, 2, 1, 3)
    k = key.reshape(b, -1, num_heads, head_dim).permute(0, 2, 1, 3)
    v = value.reshape(b, -1, num_heads, head_dim).permute(0, 2, 1, 3)

    # All views share the same intrinsic; expand to match camera counts
    Ks_q = intrinsic_out[:, :1].expand(-1, t_out, -1, -1).contiguous()
    Ks_kv = intrinsic_in[:, :1].expand(-1, t_in, -1, -1).contiguous()
    pos_enc_cross._precompute_and_cache_apply_fns(
        w2cs=p_out, Ks=Ks_q,
        w2cs_kv=p_in, Ks_kv=Ks_kv,
    )

    if self.d_proj is not None:
        raw_d = self.d_proj(hidden_states) # Q depths: (b, t_out*q_patches, 2)
        # For KV side, encoder_hidden_states may have different dim (768 for CLIP)
        # so we use default depth values with large uncertainty
        raw_d_kv_shape = (b, k.shape[2], 2)
        raw_d_kv = torch.zeros(raw_d_kv_shape, device=hidden_states.device, dtype=hidden_states.dtype)
        raw_d_kv[:, :, 0] = 0.0 # log_depth = 0 -> depth = 1.0
        raw_d_kv[:, :, 1] = 3.0 # large sigma -> large uncertainty
    else:
        raw_d = None
        raw_d_kv = None

    if self.capture_sigma and raw_d is not None:
        self.last_sigma = (raw_d[:, :, 1].detach().cpu().float(), t_out, num_patch_w)

    out = pos_enc_cross(q, k, v, predicted_d=raw_d, predicted_d_kv=raw_d_kv)
    hidden_states = out.permute(0, 2, 1, 3).reshape(b, -1, num_heads * head_dim)

```

```

if pos_enc is not None:
    # (b, seq, d) -> (b, heads, seq, head_dim)
    q = query.reshape(b, -1, num_heads, head_dim).permute(0, 2, 1, 3)
    k = key.reshape(b, -1, num_heads, head_dim).permute(0, 2, 1, 3)
    v = value.reshape(b, -1, num_heads, head_dim).permute(0, 2, 1, 3)

    # Precompute geometry (w2cs=p_out, Ks expanded to (b, t_out, 3, 3))
    # All views share the same intrinsic; expand to match t_out cameras
    Ks = intrinsic_out[:, :1].expand(-1, t_out, -1, -1).contiguous()
    pos_enc._precompute_and_cache_apply_fns(w2cs=p_out, Ks=Ks)

    # Predict depth
    if self.d_proj is not None:
        raw_d = self.d_proj(hidden_states) # (b, t_out*q_patches, 2)
    else:
        raw_d = None

    if self.capture_sigma and raw_d is not None:
        self.last_sigma = (raw_d[:, :, 1].detach().cpu().float(), t_out, num_patch_w)

    out = pos_enc(q, k, v, predicted_d=raw_d)
    # (b, heads, seq, hd) -> (b, seq, d)
    hidden_states = out.permute(0, 2, 1, 3).reshape(b, -1, num_heads * head_dim)

```

Figure 8: RayRoPE forward pass in cross-attention (left) and self-attention (right). Both predict per-token depth and uncertainty, apply RoPE to query/key features, and capture σ for analysis. Cross-attention uses default depth on the key side (CLIP-encoded references).

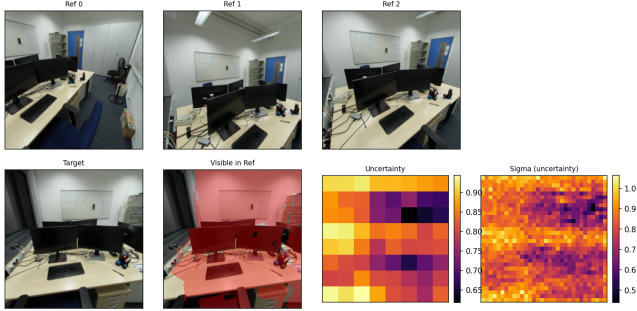


Figure 9: Covisibility analysis on ScanNet++. Three reference views (top), target view with visibility mask (red = visible in ≥ 1 reference), and uncertainty heatmap. Unobserved regions (whiteboard, left wall) receive high σ while the desk area visible in all references shows low uncertainty.



Figure 10: Uncertainty visualizations on DL3DV test scenes. Each panel shows (left to right): reference views, ground-truth target, predicted target, and uncertainty heatmap. High uncertainty (yellow) consistently aligns with regions unobserved in the references.

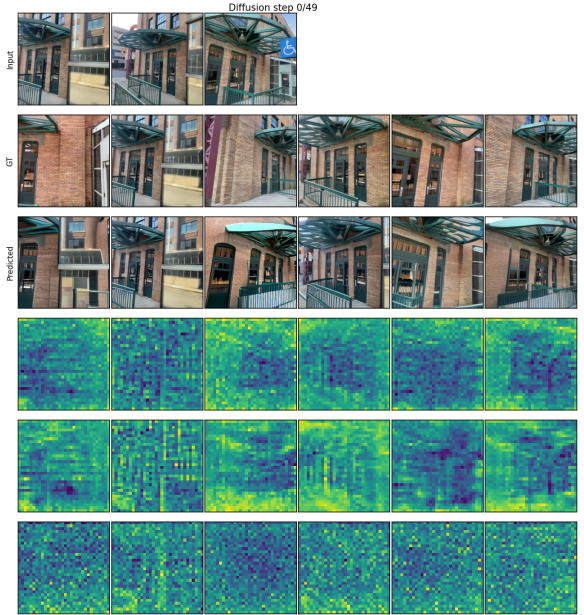


Figure 11: Multi-layer uncertainty visualization. Rows show input, ground truth, prediction, and uncertainty maps from different attention layers (cross-attn a2, average cross-attn, self-attn a2). Cross-attention layers produce more spatially coherent uncertainty maps than self-attention.